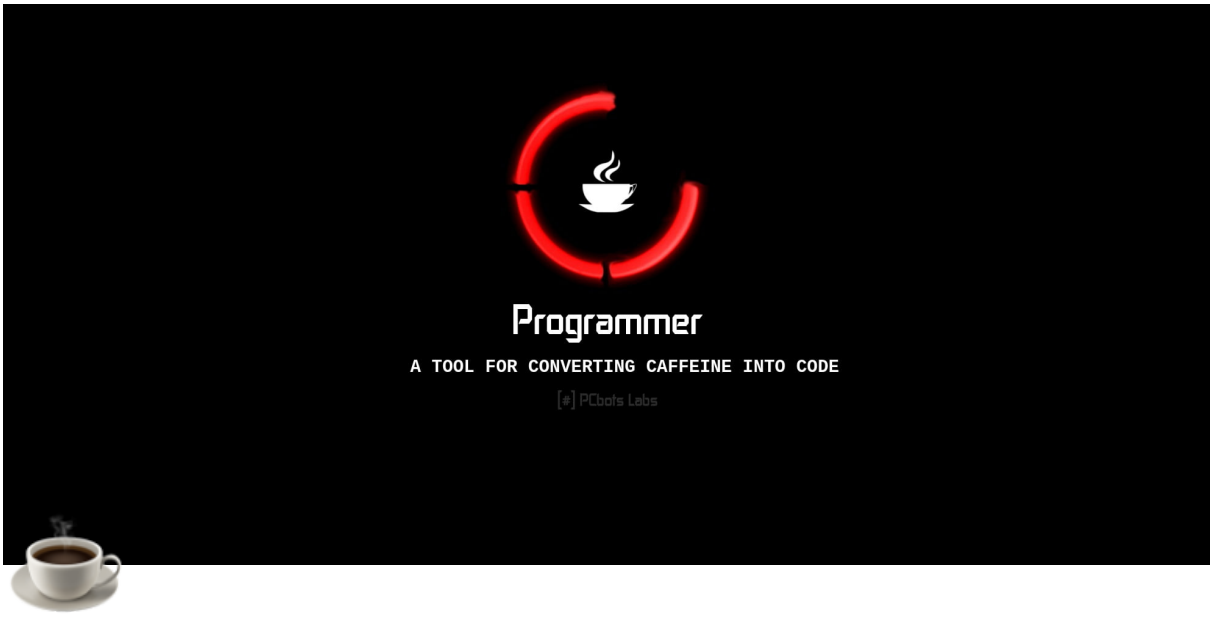

Java For beginners

This is cheat sheet prepared for those who have already knows programming and wants to learn Java.

This sheets covers basic to intermediate concepts such as getting input from the user to vectors and strings .

Basically learn java in one day.

- By Smukx

Basic Structure of a Java Program

A simple Java program looks like this:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

- `public class HelloWorld`: Defines a public class named `HelloWorld`.

- `public static void main(String[] args)`: The entry point of the program. This method is called when the program starts.
- `System.out.println("Hello, World!");`: Prints the text "Hello, World!" to the console.

Basic Input/Output

Output: `System.out.println()`

- `System.out.println(...)`: Prints the argument passed to it and then terminates the line.
- `System.out.print(...)`: Prints the argument passed to it without terminating the line.
- `System.out.printf(...)`: Allows formatted output, similar to `printf` in C.

Example:

```
System.out.println("This is a line.");
System.out.print("This is ");
System.out.print("the same line.");
System.out.printf("Number: %d, String: %s\n", 10, "exampl
e");
```

Input: `Scanner`

To read input from the user, you can use the `Scanner` class.

Example:

```
import java.util.Scanner;

public class InputExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter your name: ");
        String name = scanner.nextLine();  // Reads a line
of text
```

```
        System.out.println("Enter your age: ");
        int age = scanner.nextInt();  // Reads an integer

        System.out.println("Hello, " + name + "! You are "
+ age + " years old.");
    }
}
```

- `Scanner scanner = new Scanner(System.in);`: Creates a new `Scanner` object to read input from the console.
- `scanner.nextLine();`: Reads a full line of input as a `String`.
- `scanner.nextInt();`: Reads the next token of input as an `int`.

Types of Input methods

Method	Description
`nextBoolean()`	Reads a `boolean` value from the user
`nextByte()`	Reads a `byte` value from the user
`nextDouble()`	Reads a `double` value from the user
`nextFloat()`	Reads a `float` value from the user
`nextInt()`	Reads a `int` value from the user
`nextLine()`	Reads a `String` value from the user
`nextLong()`	Reads a `long` value from the user
`nextShort()`	Reads a `short` value from the user

Java math library

Java's `java.lang.Math` class provides a comprehensive set of methods and constants for performing mathematical operations. Since `java.lang` is imported by default, you can use the `Math` class methods without any additional import statements.

Using the Math Class

Here's a brief overview of some commonly used methods and constants in the `Math` class:

Common Constants:

- `Math.PI`: The value of π (pi), approximately 3.14159.
- `Math.E`: The value of e (Euler's number), approximately 2.71828.

Common Methods:

1. **Basic Arithmetic:**
   - `Math.abs(x)`: Returns the absolute value of `x`.
   - `Math.max(a, b)`: Returns the larger of `a` and `b`.
   - `Math.min(a, b)`: Returns the smaller of `a` and `b`.
2. **Power and Exponential:**
   - `Math.pow(a, b)`: Returns `a` raised to the power of `b` (a^b).
   - `Math.sqrt(x)`: Returns the square root of `x`.
   - `Math.cbrt(x)`: Returns the cube root of `x`.
   - `Math.exp(x)`: Returns e^x.
3. **Logarithms:**
   - `Math.log(x)`: Returns the natural logarithm (base e) of `x`.
   - `Math.log10(x)`: Returns the logarithm base 10 of `x`.
   - `Math.log1p(x)`: Returns the natural logarithm of 1+x.
4. **Trigonometric Functions:**
   - `Math.sin(x)`: Returns the sine of `x` (in radians).
   - `Math.cos(x)`: Returns the cosine of `x` (in radians).
   - `Math.tan(x)`: Returns the tangent of `x` (in radians).
   - `Math.asin(x)`: Returns the arc sine of `x` (in radians).
   - `Math.acos(x)`: Returns the arc cosine of `x` (in radians).
   - `Math.atan(x)`: Returns the arc tangent of `x` (in radians).
5. **Rounding and Remainder:**

- `Math.round(x)`: Returns the closest long to the argument.
- `Math.ceil(x)`: Returns the smallest integer that is greater than or equal to `x`.
- `Math.floor(x)`: Returns the largest integer that is less than or equal to `x`.
- `Math.rint(x)`: Returns the double value that is closest to `x` and is equal to a mathematical integer.
- `Math.random()`: Returns a random double value between 0.0 and 1.0.

Examples

Here are a few examples of how to use these methods:

```
public class MathExample {
    public static void main(String[] args) {
        // Basic Arithmetic
        int absValue = Math.abs(-10);  // 10
        int maxVal = Math.max(10, 20); // 20
        int minVal = Math.min(10, 20); // 10

        // Power and Exponential
        double power = Math.pow(2, 3); // 8.0
        double sqrt = Math.sqrt(16);   // 4.0
        double exp = Math.exp(1);      // 2.718281828459045

        // Trigonometric Functions
        double sinVal = Math.sin(Math.PI / 2); // 1.0
        double cosVal = Math.cos(0);           // 1.0
        double tanVal = Math.tan(Math.PI / 4); // 1.0

        // Logarithms
        double logVal = Math.log(10);    // 2.3025850929940
46
        double log10Val = Math.log10(10); // 1.0

        // Rounding and Remainder
        long roundVal = Math.round(2.5);   // 3
        double ceilVal = Math.ceil(2.1);   // 3.0
        double floorVal = Math.floor(2.9); // 2.0
```

```
        // Output results
        System.out.println("absValue: " + absValue);
        System.out.println("maxVal: " + maxVal);
        System.out.println("minVal: " + minVal);
        System.out.println("power: " + power);
        System.out.println("sqrt: " + sqrt);
        System.out.println("exp: " + exp);
        System.out.println("sinVal: " + sinVal);
        System.out.println("cosVal: " + cosVal);
        System.out.println("tanVal: " + tanVal);
        System.out.println("logVal: " + logVal);
        System.out.println("log10Val: " + log10Val);
        System.out.println("roundVal: " + roundVal);
        System.out.println("ceilVal: " + ceilVal);
        System.out.println("floorVal: " + floorVal);
    }
}
```

The Integer Class

The `java.lang.Integer` class in Java is a wrapper class for the primitive type `int`. It provides a range of methods and constants for working with integers, including parsing, conversion, and basic mathematical operations. Here's an overview of what you can do with the `Integer` class:

1. Creating Integer Objects

- **`Integer(int value)`**: Creates an `Integer` object representing the specified `int` value.
- **`Integer(String s)`**: Creates an `Integer` object representing the value of the specified `String`.

Example:

```
Integer intObj1 = new Integer(10);
Integer intObj2 = new Integer("20");
```

2. Constants

- `Integer.MAX_VALUE`: The maximum value an `int` can hold (`2^31 - 1`).
- `Integer.MIN_VALUE`: The minimum value an `int` can hold (`2^31`).
- `Integer.SIZE`: The number of bits used to represent an `int` in two's complement binary form.
- `Integer.BYTES`: The number of bytes used to represent an `int`.

Example:

```
int max = Integer.MAX_VALUE; // 2147483647
int min = Integer.MIN_VALUE; // -2147483648
```

3. Parsing and Conversion

- `Integer.parseInt(String s)`: Parses the string argument as a signed decimal integer.
- `Integer.parseInt(String s, int radix)`: Parses the string argument with the specified radix (base).
- `Integer.valueOf(String s)`: Returns an `Integer` object holding the value of the specified `String`.
- `Integer.valueOf(int i)`: Returns an `Integer` object holding the value of the specified `int`.

Example:

```
int num1 = Integer.parseInt("123");        // 123
int num2 = Integer.parseInt("7B", 16);     // 123 in hexade
cimal
Integer intObj = Integer.valueOf("123");   // Integer objec
t with value 123
```

4. Conversion to Other Types

- `intValue()`: Returns the value of this `Integer` as an `int`.
- `longValue()`: Returns the value of this `Integer` as a `long`.
- `floatValue()`: Returns the value of this `Integer` as a `float`.
- `doubleValue()`: Returns the value of this `Integer` as a `double`.

Example:

```
Integer intObj = 123;
int intValue = intObj.intValue();       // 123
double doubleValue = intObj.doubleValue(); // 123.0
```

5. Comparisons

- `compareTo(Integer anotherInteger)`: Compares two `Integer` objects numerically.
- `compare(int x, int y)`: Compares two `int` values numerically.
- `equals(Object obj)`: Compares this `Integer` object to the specified object for equality.

Example:

```
Integer intObj1 = 100;
Integer intObj2 = 200;

int result = intObj1.compareTo(intObj2); // -1 (100 is less
than 200)
boolean isEqual = intObj1.equals(intObj2); // false
int comparison = Integer.compare(100, 200); // -1
```

6. Conversions to Strings and Hash Codes

- `toString()`: Returns a string representation of the `Integer`.
- `hashCode()`: Returns a hash code for the `Integer`.

Example:

```
Integer intObj = 123;
String str = intObj.toString(); // "123"
int hashCode = intObj.hashCode(); // 123 (same as int valu
e)
```

7. Bit Manipulation Methods

- `bitCount(int i)`: Returns the number of one-bits in the two's complement binary representation of the specified `int` value.

- `highestOneBit(int i)`: Returns an `int` value with at most a single one-bit, in the position of the highest-order ("leftmost") one-bit in the specified `int` value.
- `lowestOneBit(int i)`: Returns an `int` value with at most a single one-bit, in the position of the lowest-order ("rightmost") one-bit in the specified `int` value.
- `numberOfLeadingZeros(int i)`: Returns the number of zero bits preceding the highest-order one-bit in the two's complement binary representation of the specified `int` value.
- `numberOfTrailingZeros(int i)`: Returns the number of zero bits following the lowest-order one-bit in the two's complement binary representation of the specified `int` value.
- `reverse(int i)`: Returns the value obtained by reversing the order of the bits in the two's complement binary representation of the specified `int` value.
- `rotateLeft(int i, int distance)`: Returns the value obtained by rotating the two's complement binary representation of the specified `int` value left by the specified number of bits.
- `rotateRight(int i, int distance)`: Returns the value obtained by rotating the two's complement binary representation of the specified `int` value right by the specified number of bits.
- `signum(int i)`: Returns the signum function of the specified `int` value.

Example:

```
int bitCount = Integer.bitCount(5); // 2 (binary: 101)
int highestBit = Integer.highestOneBit(5); // 4 (binary: 10
0)
int lowestBit = Integer.lowestOneBit(5); // 1 (binary: 001)
```

These features make the `Integer` class a powerful utility for working with integer values in Java, offering more functionality and safety compared to using the primitive `int` type directly.

also we can able to convert the interger to hex and oct too by using

- `Integer.toHexString()`

- `Integer.toOctalString()`

```
public class Main{
    public static void main(String[] args){
        int a = 20;
        System.out.println(Integer.toHexString(a)); // 14
        System.out.println(Integer.toOctalString(a)); // 24
    }
}
```

For Uppercase and lowercase

- `toUpperCase()`
- `toLowerCase()`

example

```
public class Main(){
    public static void main(String[] args){
        String text = "kavin".toUpperCase();
        String nerdy = "SMUKX".toLowerCase();
        System.out.println(text); // KAVIN
        System.out.println(nerdy); //smukx
    }
}
```

:: The output will be : KAVIN

Terminate the program in the middle !

`System.exit(0);` // Terminates the program

Loops in Java

1. Basic Loops

1.1. `for` Loop

The `for` loop is used when you know the number of iterations beforehand. It consists of three parts: initialization, condition, and increment/decrement.

Syntax:

```
for (initialization; condition; update) {
    // Code to execute
}
```

Example:

```
public class ForLoopExample {
    public static void main(String[] args) {
        for (int i = 0; i < 5; i++) {
            System.out.println("Iteration: " + i);
        }
    }
}
```

1.2. `while` Loop

The `while` loop is used when the number of iterations is not known beforehand. The loop continues as long as the condition is true.

Syntax:

```
while (condition) {
    // Code to execute
}
```

Example:

```
public class WhileLoopExample {
    public static void main(String[] args) {
        int i = 0;
        while (i < 5) {
            System.out.println("Iteration: " + i);
            i++;
```

```
        }
    }
}
```

1.3. `do-while` Loop

The `do-while` loop is similar to the `while` loop, but it guarantees at least one execution of the loop block because the condition is checked after the block is executed.

Syntax:

```
do {
    // Code to execute
} while (condition);
```

Example:

```
public class DoWhileLoopExample {
    public static void main(String[] args) {
        int i = 0;
        do {
            System.out.println("Iteration: " + i);
            i++;
        } while (i < 5);
    }
}
```

2. Advanced Loop Concepts

2.1. Enhanced `for` Loop (for-each Loop)

The enhanced `for` loop is used to iterate over arrays or collections. It's a more readable way to iterate when you don't need the index.

Syntax:

```
for (elementType element : arrayOrCollection) {
    // Code to execute
```

```
}
```

Example:

```
public class ForEachExample {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5};
        for (int number : numbers) {
            System.out.println("Number: " + number);
        }
    }
}
```

2.2. Nested Loops

Loops can be nested, meaning one loop can exist inside another. This is useful for multi-dimensional data structures like matrices.

Example:

```
public class NestedLoopExample {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

2.3. Control Statements (`break` and `continue`)

- `break`: Terminates the loop entirely.
- `continue`: Skips the current iteration and proceeds to the next one.

Example with `break` and `continue`:

```
public class BreakContinueExample {
    public static void main(String[] args) {
        for (int i = 0; i < 10; i++) {
            if (i == 5) {
                break; // Exit the loop when i equals 5
            }
            if (i % 2 == 0) {
                continue; // Skip the even numbers
            }
            System.out.println("Number: " + i);
        }
    }
}
```

2.4. Infinite Loops

An infinite loop runs indefinitely. You must use `break` or some other condition to exit the loop.

Example of an Infinite Loop:

```
public class InfiniteLoopExample {
    public static void main(String[] args) {
        int i = 0;
        while (true) {
            if (i == 5) {
                break; // Exit the loop when i equals 5
            }
            System.out.println("Number: " + i);
            i++;
        }
    }
}
```

2.5. Labelled Loops

Labelled loops allow you to specify which loop to break out of when using nested loops.

Example:

```
public class LabelledLoopExample {
    public static void main(String[] args) {
        outer: // This is a label
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (i == 1 && j == 1) {
                    break outer; // Breaks out of the outer
loop
                }
                System.out.println("i = " + i + ", j = " +
j);
            }
        }
    }
}
```

Strings in Java

1. Basic String Operations

1.1. Creating Strings

Strings can be created using string literals or the `new` keyword.

Examples:

```
String str1 = "Hello, World!";         // String literal
String str2 = new String("Hello!");    // Using new keyword
```

1.2. Basic Methods

- `length()`: Returns the length of the string.
- `charAt(int index)`: Returns the character at the specified index.

- `equals(Object obj)`: Compares this string to the specified object.
- `equalsIgnoreCase(String anotherString)`: Compares two strings, ignoring case considerations.

Examples:

```
String str = "Hello, World!";
System.out.println("Length: " + str.length());
// Length: 13
System.out.println("Character at index 1: " + str.charAt
(1)); // 'e'
System.out.println("Equality: " + str.equals("Hello"));
// false
System.out.println("Ignore case equality: " + str.equalsIgn
oreCase("HELLO, WORLD!")); // true
```

2. Intermediate String Operations

2.1. Substring and Substrings Manipulation

- `substring(int beginIndex, int endIndex)`: Returns a substring from `beginIndex` to `endIndex - 1`.
- `split(String regex)`: Splits the string into an array based on the given regular expression.

Examples:

```
String str = "Hello, World!";
String sub = str.substring(0, 5); // "Hello"

String[] parts = str.split(", "); // ["Hello", "World!"]
```

2.2. String Concatenation

- **Using `+` Operator**: Concatenates strings.
- **Using `concat(String str)`**: Concatenates the specified string.

Examples:

```
String str1 = "Hello";
String str2 = "World!";
String combined = str1 + ", " + str2; // "Hello, World!"
String combined2 = str1.concat(", ").concat(str2); // "Hell
o, World!"
```

2.3. Case Conversion and Trimming

- `toUpperCase()` / `toLowerCase()`: Converts all characters to upper/lower case.
- `trim()`: Removes leading and trailing whitespace.

Examples:

```
String str = "   Hello, World!   ";
System.out.println("Trimmed: " + str.trim()); // "Hello, Wo
rld!"
System.out.println("Uppercase: " + str.toUpperCase()); // "
HELLO, WORLD!   "
System.out.println("Lowercase: " + str.toLowerCase()); // "
hello, world!   "
```

2.4. Replacement and Modification

- `replace(char oldChar, char newChar)`: Replaces all occurrences of `oldChar` with `newChar`.
- `replaceAll(String regex, String replacement)`: Replaces each substring of this string that matches the given regular expression with the given replacement.

Examples:

```
String str = "Hello, World!";
String replaced = str.replace('o', 'a'); // "Hella, Warld!"
String replacedAll = str.replaceAll("o", "a"); // "Hella, W
arld!"
```

3. Advanced String Operations

3.1. StringBuilder and StringBuffer

For scenarios where you need to modify strings frequently, use `StringBuilder` (not thread-safe) or `StringBuffer` (thread-safe) for better performance due to their mutable nature.

Example:

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(", World!");
System.out.println(sb.toString()); // "Hello, World!"

// StringBuffer (thread-safe)
StringBuffer sbf = new StringBuffer("Hello");
sbf.append(", World!");
System.out.println(sbf.toString()); // "Hello, World!"
```

3.2. Regular Expressions

Regular expressions (regex) are powerful for pattern matching and text manipulation.

- `matches(String regex)`: Tells whether the string matches the given regular expression.
- `replaceAll(String regex, String replacement)`: Replaces each substring that matches the regex with the replacement.

Example:

```
String str = "abc123xyz";
boolean matches = str.matches("[a-z]+\\\\d+[a-z]+"); // tru
e

String cleaned = str.replaceAll("\\\\d", ""); // "abcxyz"
(removes digits)
```

3.3. Formatting and Parsing

- `String.format(String format, Object... args)`: Returns a formatted string using the specified format string and arguments.

Example:

```
String formatted = String.format("Name: %s, Age: %d", "Joh
n", 25); // "Name: John, Age: 25"
```

3.4. Handling Unicode and Special Characters

Java strings are Unicode, so they can handle special characters and emojis.

Example:

```
String emoji = "😊";
System.out.println("Emoji: " + emoji); // 😊

String unicode = "\\u263A"; // Unicode for ☺
System.out.println("Unicode: " + unicode); // ☺
```

3.5. String Comparison and Sorting

- `compareTo(String anotherString)`: Compares two strings lexicographically.
- `compareToIgnoreCase(String str)`: Compares two strings lexicographically, ignoring case differences.

Example:

```
String str1 = "apple";
String str2 = "banana";

int result = str1.compareTo(str2); // Negative value (apple
is less than banana)
int resultIgnoreCase = str1.compareToIgnoreCase("Banana");
// Negative value
```

3.6. Real-World Problem Examples

Example 1: Count Occurrences of a Character

```
public class CountCharacterOccurrences {
    public static void main(String[] args) {
        String str = "hello world";
```

```
        char target = 'l';
        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            if (str.charAt(i) == target) {
                count++;
            }
        }
        System.out.println("Occurrences of 'l': " + count);
// 3
    }
}
```

Example 2: Reverse a String

```
public class ReverseString {
    public static void main(String[] args) {
        String str = "hello";
        String reversed = new StringBuilder(str).reverse().
toString();
        System.out.println("Reversed: " + reversed); // "ol
leo"
    }
}
```

Example 3: Palindrome Check

```
public class PalindromeCheck {
    public static void main(String[] args) {
        String str = "racecar";
        String reversed = new StringBuilder(str).reverse().
toString();
        if (str.equals(reversed)) {
            System.out.println(str + " is a palindrome.");
        } else {
            System.out.println(str + " is not a palindrom
e.");
        }
```

```
    }
}
```

Arrays / Vectors in Java

1. Arrays in Java

1.1. Basics of Arrays

Arrays are a fixed-size data structure that stores elements of the same type. The size of an array is defined at the time of its creation and cannot be changed later.

Creation and Initialization:

- **Syntax:** `type[] arrayName = new type[arraySize];`
- **Example:**

```
int[] intArray = new int[5]; // Declares an array of int
egers with size 5
String[] stringArray = {"Hello", "World"}; // Initialize
s an array with elements
```

Accessing and Modifying Elements:

- Access elements using the index, starting from 0.
- Modify elements by assigning new values to specific indices.

Example:

```
int[] numbers = {1, 2, 3, 4, 5};
System.out.println(numbers[0]); // Output: 1
numbers[0] = 10;
System.out.println(numbers[0]); // Output: 10
```

1.2. Intermediate Array Concepts

- **Multidimensional Arrays:** Arrays of arrays, useful for representing matrices or grids.

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
System.out.println(matrix[1][1]); // Output: 5
```

- **Iterating Through Arrays:** Using loops, typically `for` or enhanced `for` loops.

```
int[] arr = {1, 2, 3, 4, 5};
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
// Enhanced for loop
for (int num : arr) {
    System.out.println(num);
}
```

- **Common Methods (Arrays Class):** The `java.util.Arrays` class provides utility methods for arrays.
  - `Arrays.toString()`: Converts the array to a string representation.
  - `Arrays.sort()`: Sorts the array.
  - `Arrays.equals()`: Compares two arrays.

  Example:

```
import java.util.Arrays;

int[] arr = {3, 1, 4, 1, 5};
Arrays.sort(arr);
System.out.println(Arrays.toString(arr)); // Output: [1,
1, 3, 4, 5]
```

1.3. Advanced Array Concepts

- **Array Manipulation:** Involves operations like shifting, rotation, and resizing (usually by creating a new array and copying elements).
- **Dynamic Array Alternatives:** Since arrays have a fixed size, dynamic data structures like `ArrayList` are often used when the size needs to change frequently.
- **Memory Considerations:** Arrays are stored in contiguous memory locations. This can lead to efficient data access but may also cause memory fragmentation if the array size is large.

Example of Dynamic Resizing:

```
int[] original = {1, 2, 3};
int[] resized = new int[original.length + 2];
System.arraycopy(original, 0, resized, 0, original.length);
resized[3] = 4;
resized[4] = 5;
System.out.println(Arrays.toString(resized)); // Output:
[1, 2, 3, 4, 5]
```

2. Vectors in Java

Vectors are part of the `java.util` package and implement the `List` interface. Unlike arrays, vectors are dynamic and can grow or shrink in size. They are synchronized, making them thread-safe but potentially less efficient than other list implementations.

2.1. Basics of Vectors

Creation and Initialization:

- **Syntax:** `Vector<Type> vectorName = new Vector<Type>();`
- **Example:**

```
Vector<Integer> vector = new Vector<>();
vector.add(1);
vector.add(2);
```

Accessing and Modifying Elements:

- Access elements using the `get` method.
- Modify elements using the `set` method.

Example:

```
Vector<String> vector = new Vector<>();
vector.add("Hello");
vector.add("World");
System.out.println(vector.get(0)); // Output: Hello
vector.set(1, "Java");
System.out.println(vector.get(1)); // Output: Java
```

2.2. Intermediate Vector Concepts

- **Dynamic Sizing:** Vectors automatically resize themselves when elements are added or removed. The capacity grows by doubling itself when needed, ensuring amortized constant time complexity for insertion.
- **Iteration:** Similar to arrays, vectors can be iterated using loops.

  ```
  Vector<String> vector = new Vector<>();
  vector.add("A");
  vector.add("B");
  vector.add("C");

  for (String s : vector) {
      System.out.println(s);
  }
  ```

- **Common Methods:**
  - `addElement(E obj)`: Adds an element to the end.
  - `removeElement(Object obj)`: Removes the first occurrence of the object.
  - `size()`: Returns the number of elements.
  - `capacity()`: Returns the current capacity.

  Example:

```
Vector<Integer> vector = new Vector<>();
vector.add(10);
vector.add(20);
vector.add(30);
vector.remove((Integer) 20); // Removes element 20
System.out.println(vector.size()); // Output: 2
System.out.println(vector.capacity()); // Output: Initia
l capacity (default is 10)
```

2.3. Advanced Vector Concepts

- **Synchronization and Thread Safety:** Vectors are synchronized, making them thread-safe. This is achieved by synchronizing methods, which can lead to performance overhead. For better performance in a single-threaded context, consider using `ArrayList`.
- **Enumeration and Iterators:** Vectors support both `Enumeration` and `Iterator` interfaces for traversing elements. `Enumeration` is an older approach, while `Iterator` is more modern and provides methods like `remove`.

  Example:

  ```
  Vector<String> vector = new Vector<>();
  vector.add("X");
  vector.add("Y");
  vector.add("Z");

  Enumeration<String> enumeration = vector.elements();
  while (enumeration.hasMoreElements()) {
      System.out.println(enumeration.nextElement());
  }
  ```

- **Conversion to Other Collections:** Vectors can be converted to other collections like `ArrayList` or arrays.

  Example:

  ```
  Vector<String> vector = new Vector<>();
  vector.add("One");
  vector.add("Two");
  ```

```
// Convert to ArrayList
ArrayList<String> arrayList = new ArrayList<>(vector);

// Convert to Array
String[] array = vector.toArray(new String[0]);
```

- **Legacy Nature:** Vectors are considered a legacy class. Although they are still used, modern applications often prefer `ArrayList` for non-thread-safe implementations and `CopyOnWriteArrayList` or other concurrent collections for thread-safe implementations.